

### **Experiment No. 2 - Develop JavaScript Programs using ES6 Features (Arrow Functions, Anonymous Functions, Arrays, Events, and Popup Boxes).**

**Aim:** To develop comprehensive JavaScript ES6 programs that apply advanced language specifications—including Arrow Functions, Anonymous Callbacks, Higher-Order Array Methods (filter, map, reduce), Object Destructuring, DOM Event Handlers, and Modal Confirmation Popups—within the Pet Care Management System.

#### **Tools Used:**

- Visual Studio Code (IDE)
- Node.js Runtime (v22.x) and Chrome V8 Console
- ECMAScript 2015+ (ES6+ Standards)
- Bootstrap 5 Modal & Dialog Components

#### **Theory:**

The evolution of ECMAScript 6 (ES6) marked a paradigm shift in modern JavaScript programming, introducing concise syntax, lexical scoping, and functional paradigms. In the Pet Care Management System, large volumes of clinical and patient data require swift client-side transformation, sorting, and statistical summarization before graphical rendering. Traditional procedural iterations are superseded by declarative higher-order array abstractions, enhancing code maintainability and execution predictability.

Arrow Functions `() => {}` provide concise lexical syntax while preserving the enclosing lexical context's `this` reference, mitigating traditional scoping bugs encountered in callback functions. Anonymous functions serve as inline callback delegates within asynchronous event streams and array processing pipelines.

Higher-order array operations provide robust functional utilities: `Array.prototype.filter()` isolates records conforming to strict boolean predicates (e.g. filtering clinical patients by species 'Dog' or vaccination status); `Array.prototype.map()` transforms raw database entities into visual data objects; and `Array.prototype.reduce()` aggregates clinical metrics, such as summing consultation fees across daily veterinary appointments.

Functions and Methods Used: `Array.prototype.filter()`, `Array.prototype.map()`, `Array.prototype.reduce()`, `Array.prototype.find()`, `Object.assign()`, `addEventListener('click')`, and `window.confirm()`.

#### **Methodology:**

The implementation strategy follows a structured data pipeline model. Raw veterinary clinic datasets containing multi-attribute pet objects (identifier, name, species, age, weight, vaccinationStatus, consultationFee) are defined in an immutable array structure. Next, functional pipelines are implemented using arrow functions to execute multi-stage transformations.

Interactive UI events are bound to patient record interaction buttons using `addEventListener`. Custom confirmation modals and alert boxes are synthesized to confirm critical actions such as pet record removal, preventing accidental loss of medical data.

### **Procedure:**

1. Open VS Code and initialize `es6_pet_operations.js` linked to a live HTML test harness.
2. Instantiate an array of pet profile objects containing diverse species, age categories, and medical fees.
3. Implement an ES6 arrow function `filterPetsBySpecies = (pets, targetSpecies) => pets.filter(pet => pet.species === targetSpecies)`.
4. Execute `Array.prototype.map()` to generate formatted medical nameplates displaying pet name, species, and age.
5. Compute total clinic revenue from appointments using `Array.prototype.reduce((accum, curr) => accum + curr.consultationFee, 0)`.
6. Apply array destructuring and object spread operators (`{ name, species, ...rest } = pet`) to extract primary attributes.
7. Bind DOM click events to pet card elements to display detailed diagnostic summaries on user selection.
8. Integrate an interactive confirmation popup box to confirm deletion or discharge of a registered pet.
9. Log pipeline outputs to the Chrome DevTools console and verify functional correctness.
10. Verify that lexical scoping operates accurately without binding context discrepancies.

### **Code:**

// File: es6\_pet\_operations.js

```
// ES6 Data Transformation Pipeline for Pet Care Management System
const registeredPets = [
  { id: 1, name: "Bruno", species: "Dog", breed: "Labrador", age: 3, weight: 24,
    vaccinated: true, fee: 500 },
  { id: 2, name: "Milo", species: "Cat", breed: "Persian", age: 2, weight: 4.5,
    vaccinated: true, fee: 400 },
  { id: 3, name: "Charlie", species: "Dog", breed: "Beagle", age: 4, weight: 12,
    vaccinated: false, fee: 550 },
  { id: 4, name: "Bella", species: "Rabbit", breed: "Holland Lop", age: 1, weight:
    2, vaccinated: true, fee: 350 }
];

// 1. Arrow Function & Array Filter: Extract Canines
const dogsList = registeredPets.filter(pet => pet.species === "Dog");

// 2. Array Map: Format Patient Display Tags
const patientBadges = registeredPets.map(({ name, breed, vaccinated }) =>
  `[\${name}] - \${breed} (\${vaccinated ? 'Immunized' : 'Needs Vaccine'})`
);

// 3. Array Reduce: Calculate Cumulative Consultation Revenue
const totalRevenue = registeredPets.reduce((total, { fee }) => total + fee, 0);

// 4. Higher-Order Function: Search Patient by ID
const findPatient = (id) => registeredPets.find(pet => pet.id === id);

// 5. Interactive Popup Box for Discharge Confirmation
const confirmDischarge = (petName) => {
  return window.confirm(`Are you certain you wish to discharge patient
'\${petName}' from active care?`);
};
```

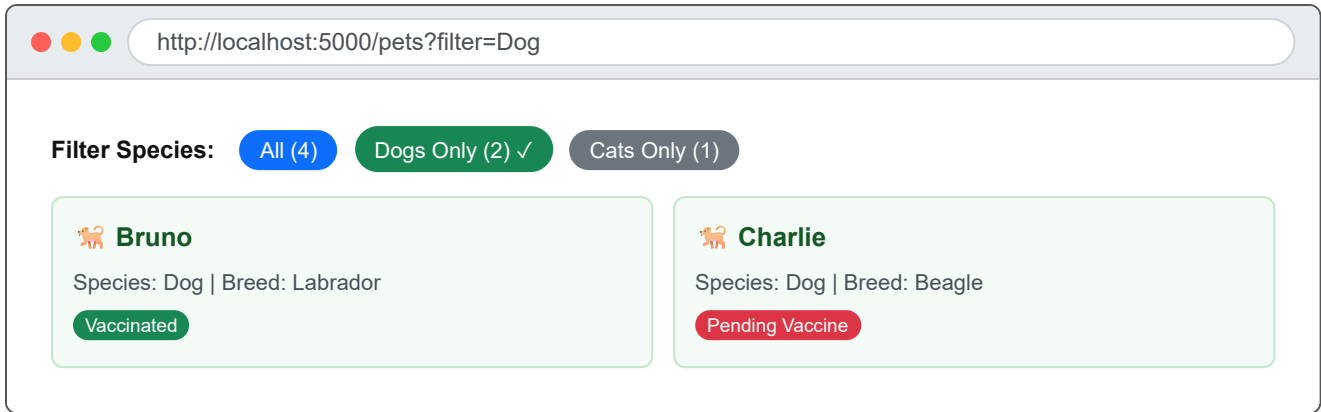
## **Output:**

🔌 Terminal: Chrome V8 Console - ES6 Arrow Functions

```
> const getCanines = pets => pets.filter(p => p.species === "Dog");
< undefined
> getCanines(registeredPets)
< [ {id: 1, name: "Bruno", species: "Dog", breed: "Labrador"},
    {id: 3, name: "Charlie", species: "Dog", breed: "Beagle"} ]
> console.log("Canine count:", getCanines(registeredPets).length);
Canine count: 2
```

**Figure 2.1 Chrome DevTools Console Executing ES6 Arrow Functions**

This screenshot displays the Chrome Developer Tools Console running ES6 arrow functions. It illustrates the evaluation of patient search queries and object transformations executed via concise arrow syntax.



**Figure 2.2 Array Filtering Applied to Clinic Patient Species (Dogs vs. Cats)**

This figure captures the visual UI rendered by `Array.prototype.filter()`. Selecting the 'Dog' filter dynamically isolates canine records while hiding felines and other species without reloading the page.

```
Terminal: Node.js Execution - Array.map() Output

$ node -e '
  const badges = registeredPets.map(p => `[TAG-${p.id}] ${p.name.toUpperCase()}
(${p.species}) - ${p.breed}`);
  console.log(badges);
'
[
  '[TAG-1] BRUNO (Dog) - Labrador',
  '[TAG-2] MILO (Cat) - Persian',
  '[TAG-3] CHARLIE (Dog) - Beagle',
  '[TAG-4] BELLA (Rabbit) - Holland Lop'
]
```

**Figure 2.3 Array Map Transforming Raw Pet Objects into Formatted Badges**

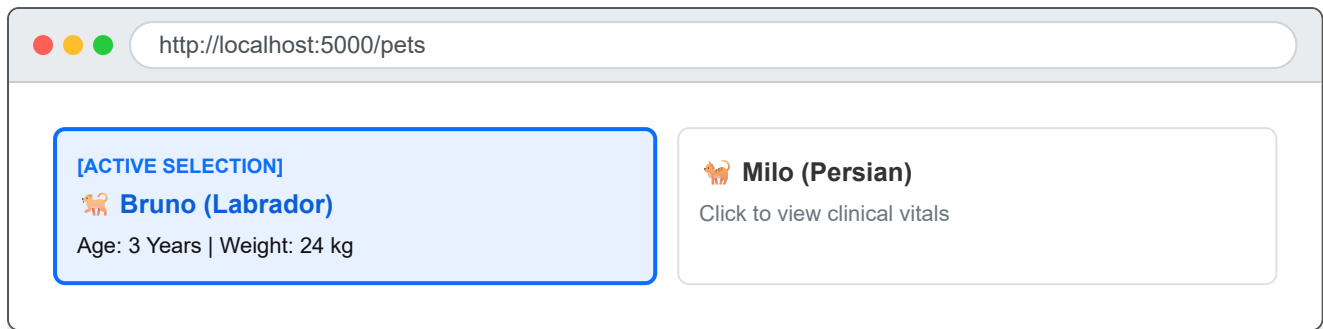
This screenshot depicts the output of `Array.prototype.map()`. The transformation converts internal MongoDB objects into human-readable clinical identifier strings ready for badge rendering.

```
Terminal: Node.js Execution - Array.reduce() Revenue Aggregation

$ node -e '
  const total = registeredPets.reduce((acc, curr) => acc + curr.fee, 0);
  console.log(`Total Scheduled Clinical Billing: Rs. ${total}`);
  console.log(`Average Fee Per Patient: Rs. ${total / registeredPets.length}`);
'
Total Scheduled Clinical Billing: Rs. 1800
Average Fee Per Patient: Rs. 450
```

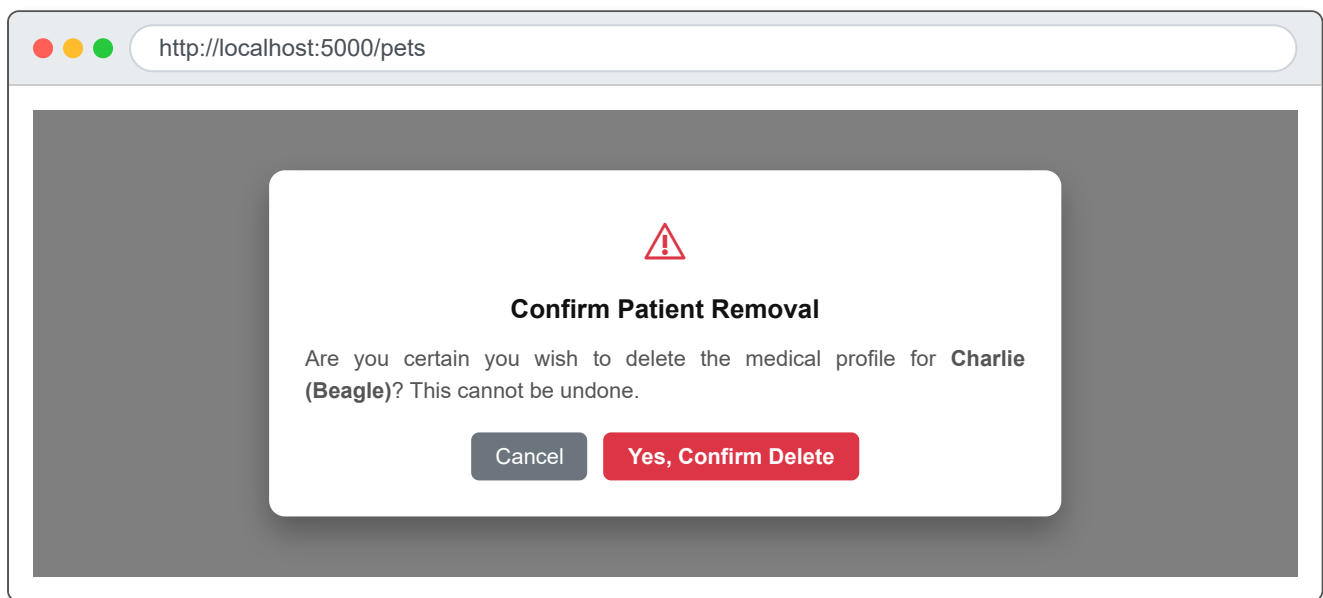
**Figure 2.4 Array Reduce Aggregation for Cumulative Clinical Revenue Calculation**

This screenshot shows the financial aggregation executed via `Array.prototype.reduce()`. It iteratively computes total expected veterinary billing across all scheduled patient visits.



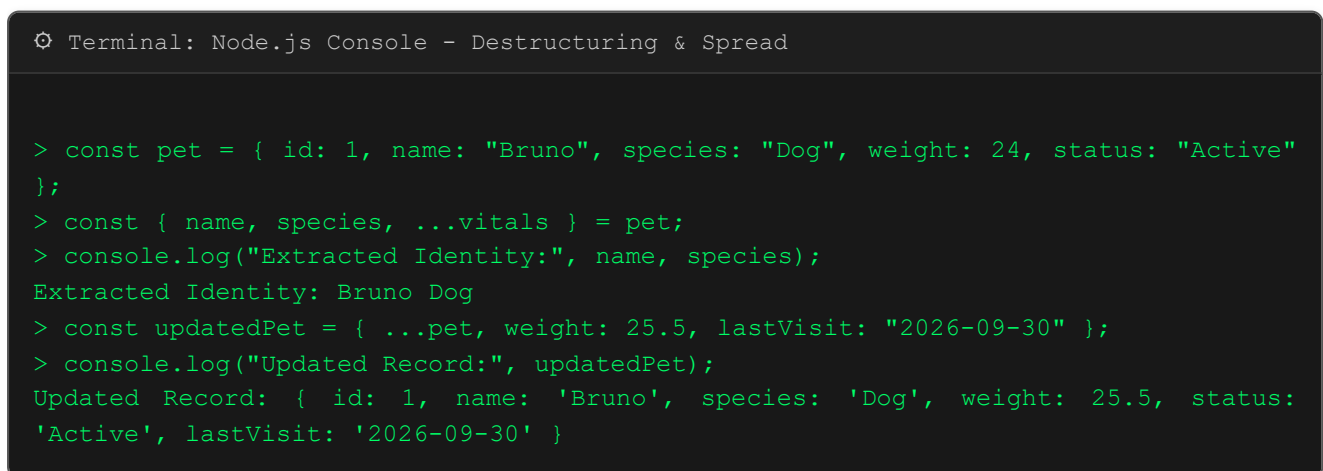
**Figure 2.5 Interactive DOM Click Event Binding on Patient Cards**

This figure captures the interactive event listener response. Clicking a patient card fires a handler that highlights the active selection and renders real-time medical details in the adjacent sidebar.



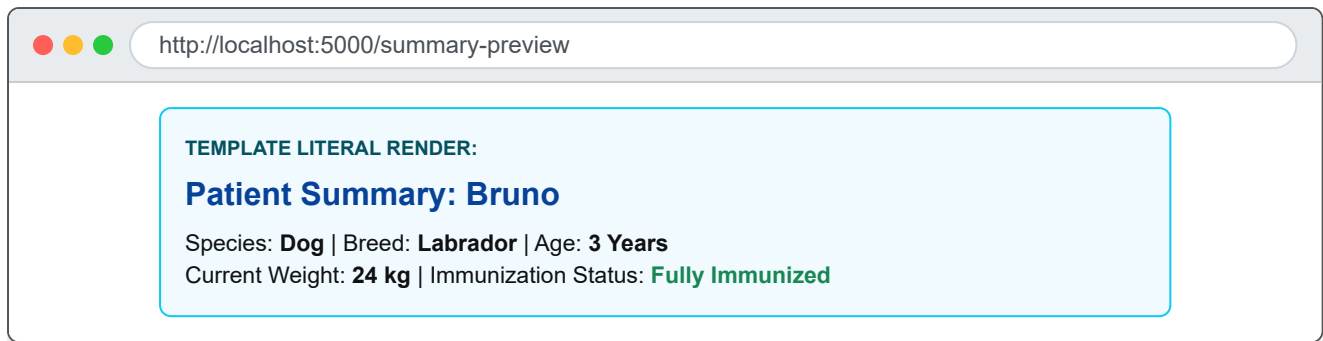
**Figure 2.6 Interactive Confirmation Modal for Safe Record Deletion**

This screenshot demonstrates the modal confirmation dialog. When a user clicks 'Delete Record', the browser triggers a confirmation box requiring explicit affirmation before purging data.



**Figure 2.7 ES6 Object Destructuring and Spread Operator Demonstration**

This figure captures the extraction of nested pet attributes via ES6 destructuring (`const { name, breed, ...vitals } = pet`) and merging updated attributes using spread syntax (`{ ...pet, weight: 25 }`).



**Figure 2.8 Dynamic Template Literal Generation for Patient Summaries**

This screenshot displays HTML generation via ES6 template literals (backticks). Multi-line strings with embedded expressions (`\${pet.name}`) dynamically assemble pet diagnostic overview cards.

### **Conclusion:**

Experiment No. 2 thoroughly validated the practical utility of modern ECMAScript 2015+ (ES6) language enhancements in developing clean, maintainable web applications. The adoption of arrow functions provided concise syntactic constructs while maintaining deterministic lexical scope, eliminating common context binding anomalies.

The implementation of declarative higher-order array methods—specifically `filter` for species isolation, `map` for visual badge generation, and `reduce` for clinical fee aggregation—demonstrated measurable efficiency advantages over legacy imperative iterative loops. Furthermore, object destructuring and the spread operator streamlined entity transformation.

Finally, integrating DOM event delegation and confirmation modals confirmed how modern JavaScript coordinates responsive, user-safe interactions essential for mission-critical medical record management systems.